Rozdział 3

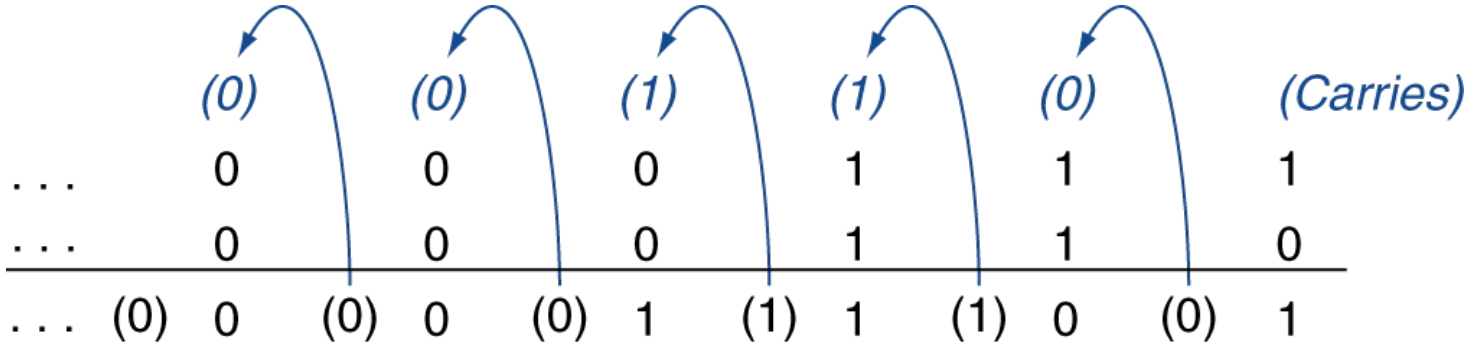
Arytmetyka komputerów

Arytmetyka dla komputerów

- Operacje na liczbach całkowitych (integers)
  - Dodawanie i odejmowanie
  - Mnożenie i dzielenie
  - Radzenie sobie z przepełnieniem (overflow)
- Liczby zmiennoprzecinkowe (FP: floating-point)
  - Reprezentacja i operacje

Dodawanie całkowite

- Przykład: 7 + 6 (carries – przeniesienia)



- Przepełnienie gdy wynik poza zakresem
  - Dodawanie operandów o przeciwnych znakach – nie ma przepełnienia
  - Dodawanie operandów dodatnich
    - Przepełnienie gdy znak wyniku jest 1
  - Dodawanie operandów ujemnych
    - Przepełnienie gdy znak wyniku jest 0

Odejmowanie całkowite

- Dodaj – drugi operand ze zmienionym znakiem
- Przykład: 7 – 6 = 7 + (–6)

+7:	0000 0000 … 0000 0111
–6:	1111 1111 … 1111 1010
+1:	0000 0000 … 0000 0001

- Przepełnienie gdy wynik poza zakresem
  - Odejmowanie operandów o tym samym znaku – nie ma przepełnienia
  - Odejmowanie liczby ujemnej od dodatniej
    - Przepełnienie gdy znak wyniku jest 0
  - Odejmowanie liczby dodatniej od ujemnej
    - Przepełnienie gdy znak wyniku jest 1

Działanie gdy przepełnienie

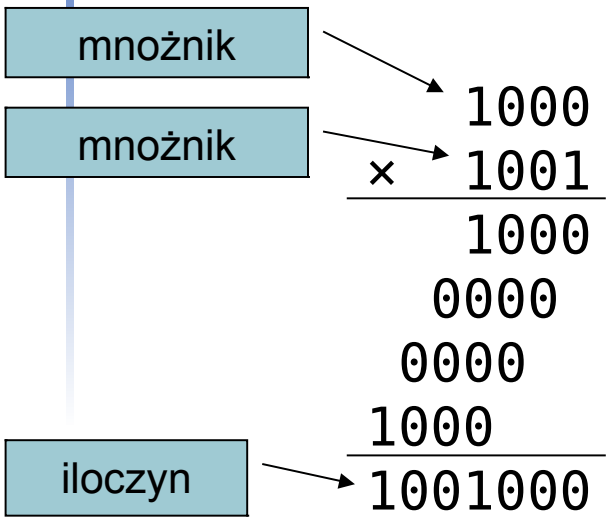
- Niektóre języki ignorują przepełnienie (C)
  - Używaj instrukcji ARM'a z polem S = 0
- Inne języki żądają wywołania wyjątku (np. Ada, Fortran)
  - Ustawiaj pole S na 1, instrukcje skoku BVS lub BVC (skok gdy bit overflow V = 1 lub 0)
  - Gdy przepełnienie wywołuj obsługę wyjątku
    - Wywołanie funkcji „naprawiającej" sytuację, własnej, lub systemowej

Arytmetyka dla multimediów

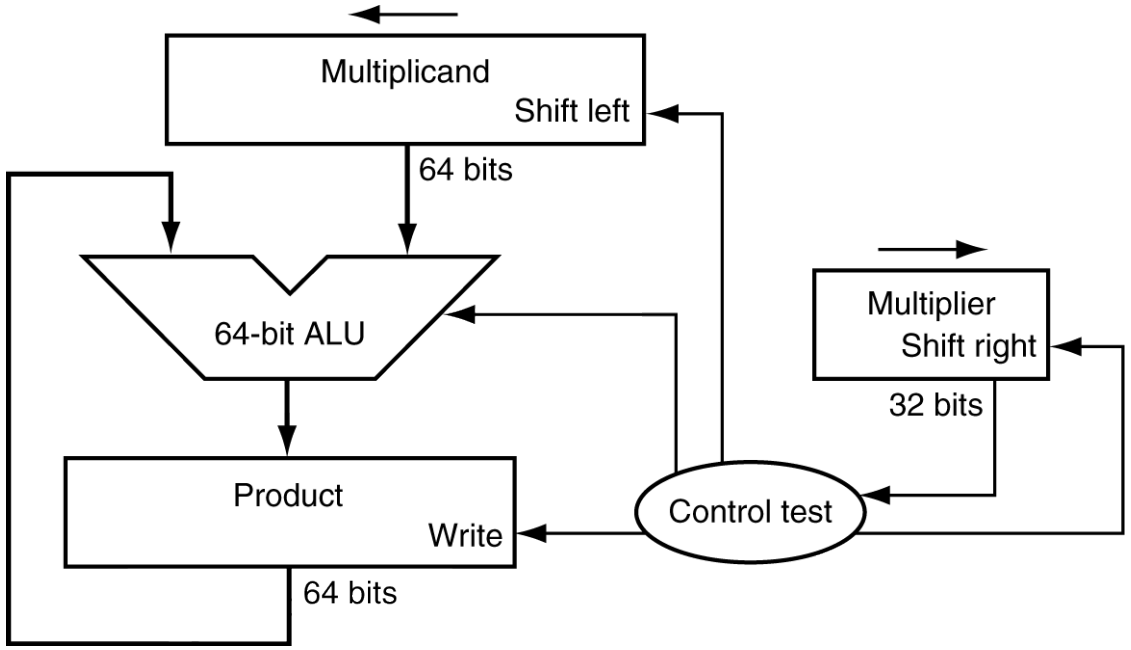
- Grafika i przetwarzanie mediów działa na wektorach danych 8- i 16-bitowych
  - Użyj 64-bitowego sumatora z podzielonym łańcuchem przeniesień
    - Działaj na 8×8-, 4×16-, lub 2×32-bitach
  - SIMD (single-instruction, multiple-data)
- Operacje z nasyceniem
  - Przy przepełnieniu przyjmij największą lub najmniejszą reprezentowalną wartość
    - Arytmetyka uzupełnienia do 2 jest modulo zakres
  - Daje obcinanie w audio, nasycenie w wideo

Mnożenie

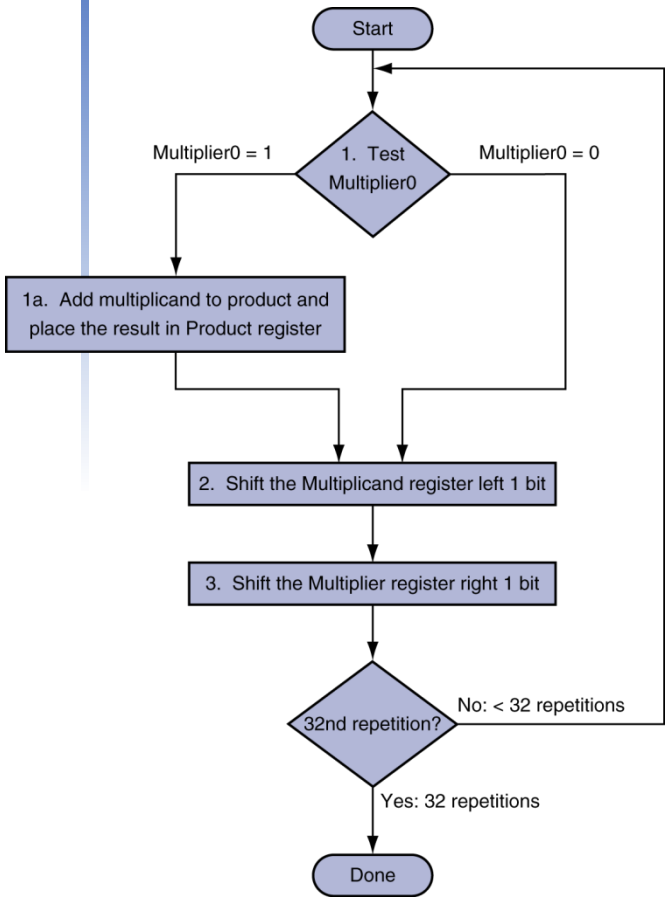
- Zacznijmy od mnożenia „pisemnego”

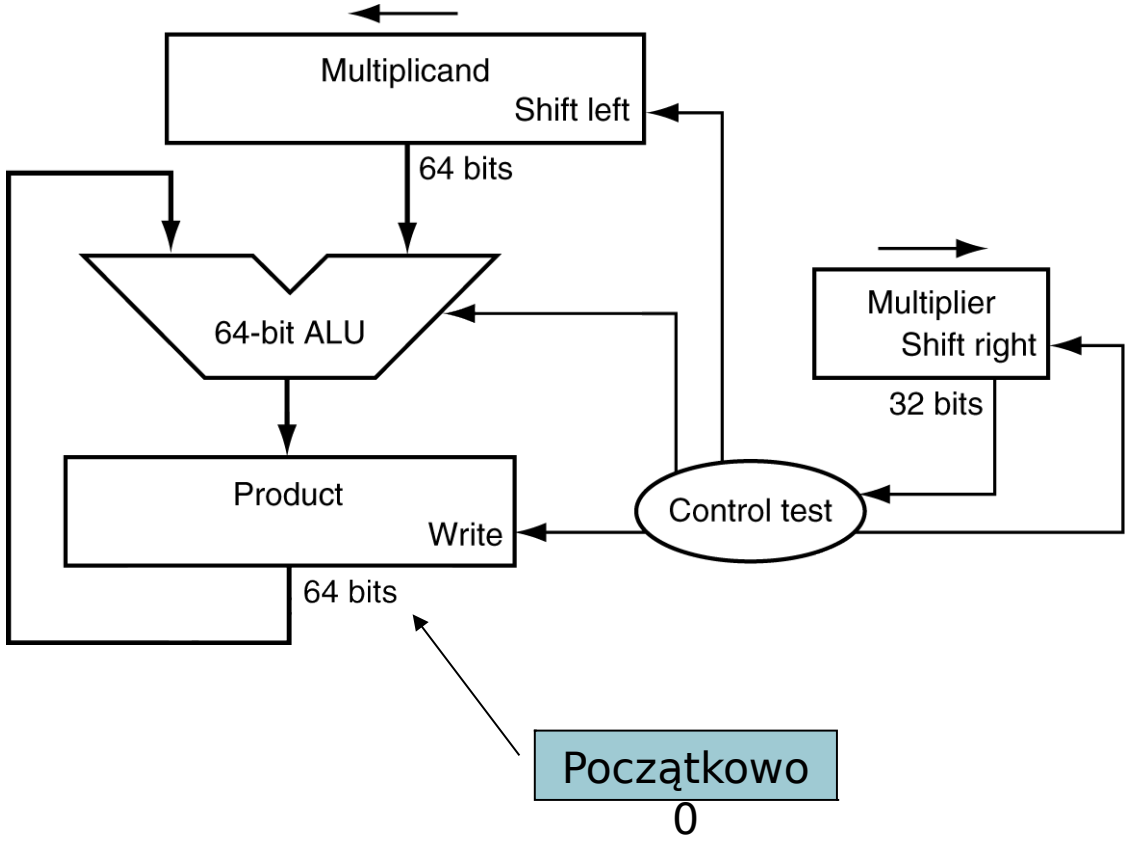


Długość iloczynu jest sumą długości operandów



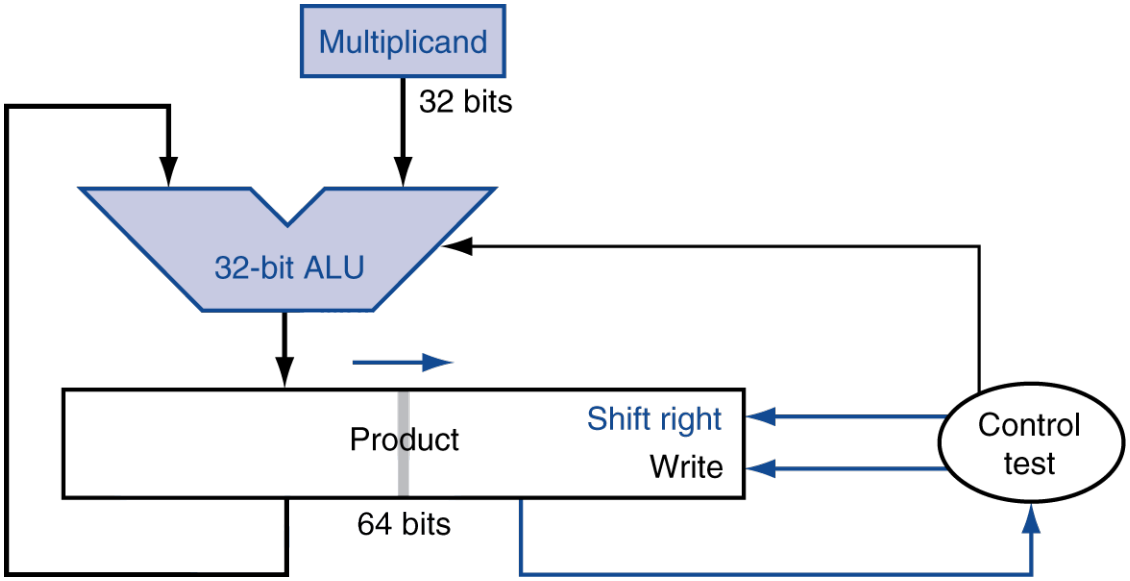
Układ mnożący





Lepszy układ mnożący

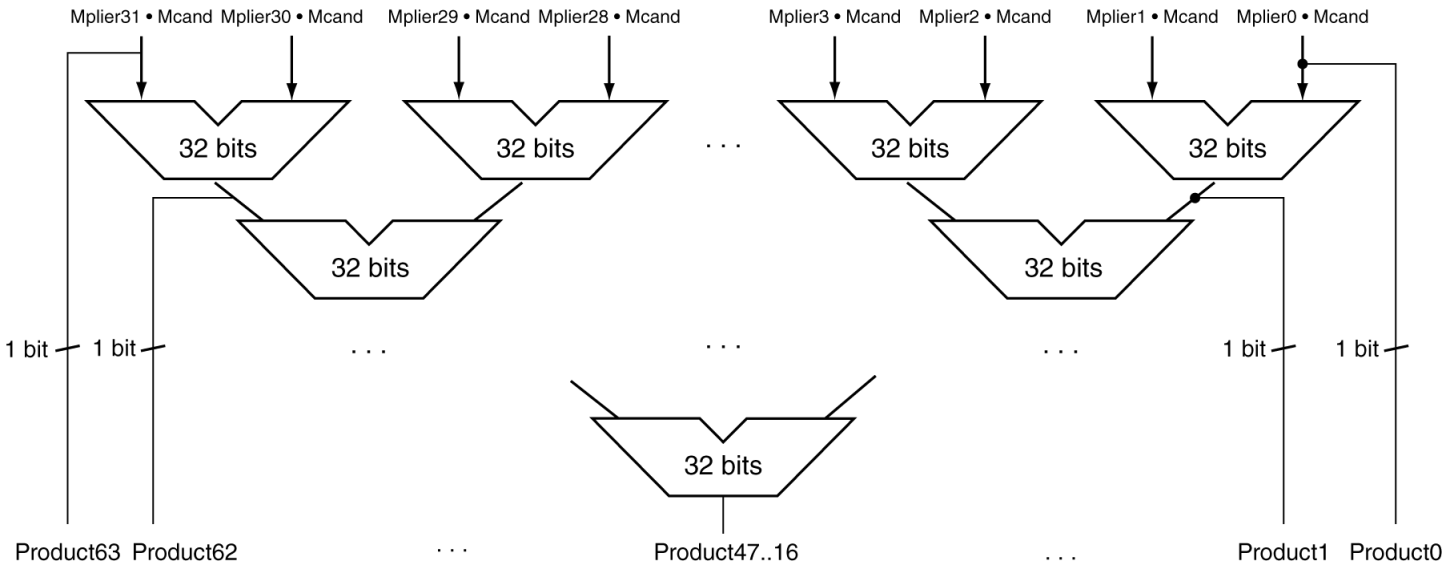
- Równoległe dodawanie z przesuwaniem



- Jeden cykl na dodawanie mnożnika
  - OK gdy częstotliwość mnożeń jest niska

Szybszy układ mnożący

- Używa wielu sumatorów
  - Wymiana koszt/wydajność

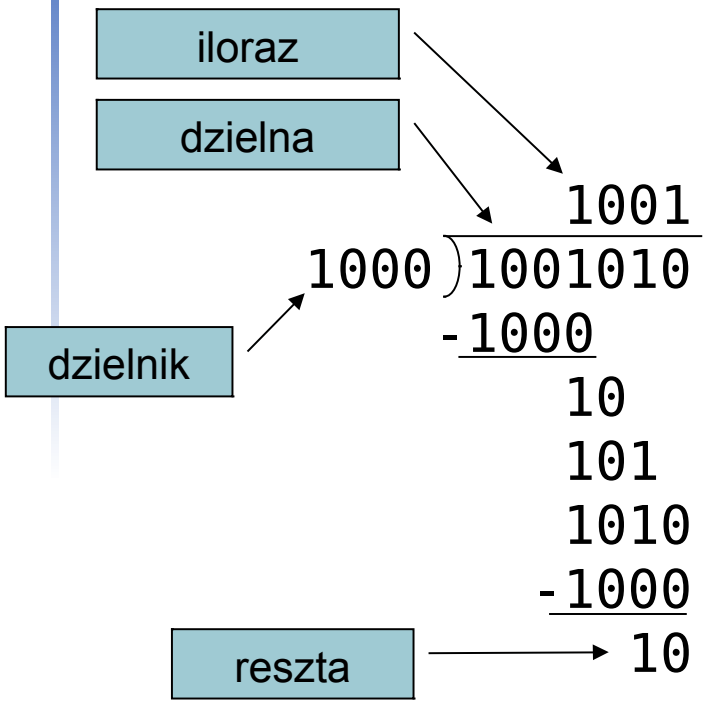


- Może być z potokiem
  - Szereg mnożeń wykonywanych równolegle

Mnożenie w ARM'ie

- Wynik – dolne 32 bity iloczynu
  - Nie ma różnicy między mnożeniem znakowym i bezznakowym
  - Nie ustawia bitu overflow (V)
- Instrukcja
  - `MUL rd, rs, rt  ; rd = (rs * rt)`$_{31:0}$
    - Rejestry rd i rs muszą się różnić
    - Rejestru rt nie można zastąpić polem immediate

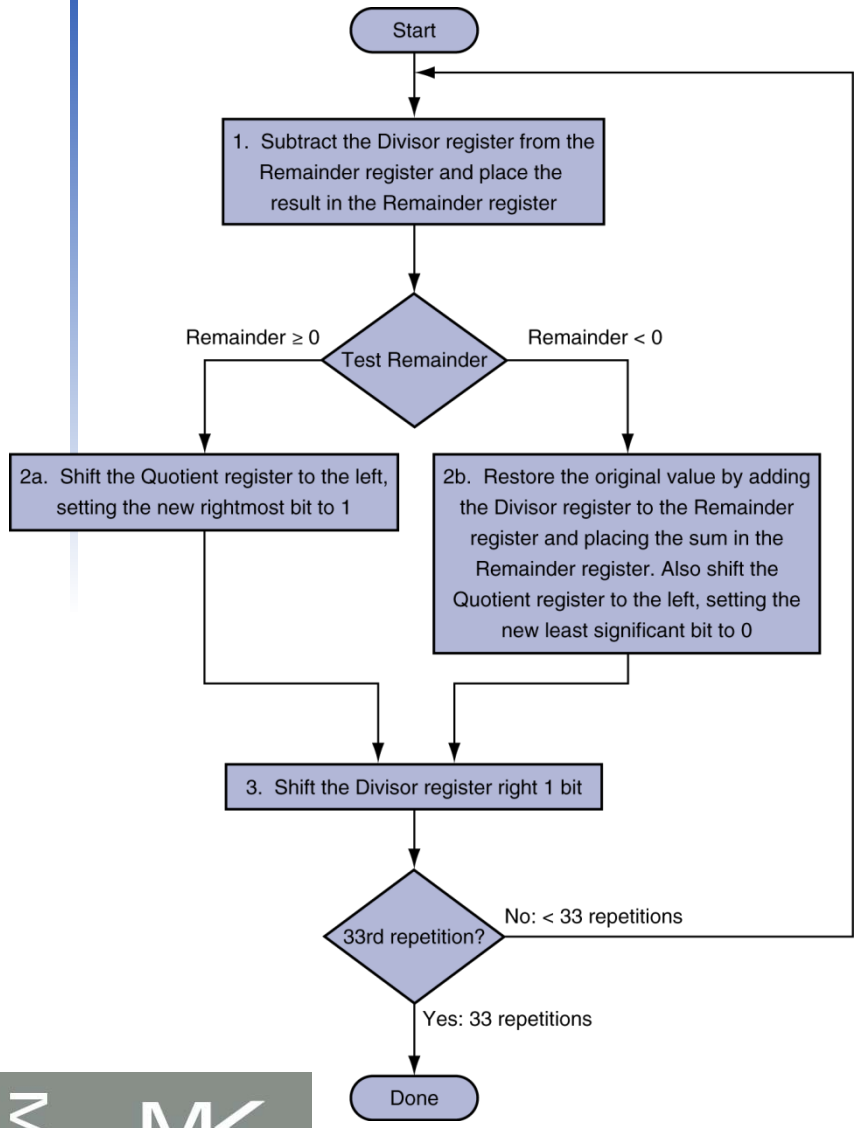
Dzielenie

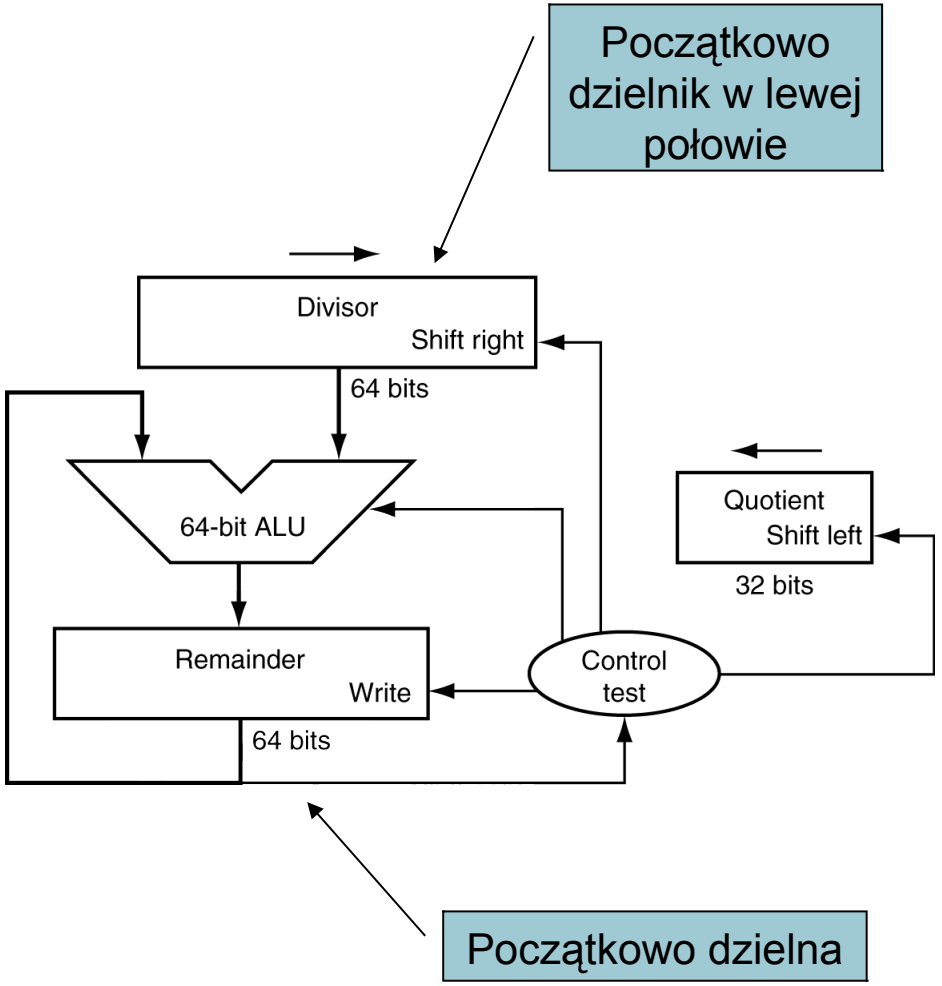


- Sprawdź czy dzielnik = 0
- Dzielenie „pisemne”
  - Jeśli dzielnik ≤ bitów dzielnej
    - 1 w ilorazie, odejmij
  - Jeśli nie
    - 0 w ilorazie, weź następny bit dzielnej
- Dzielenie bez porównywania
  - Wykonaj odejmowanie, jeśli wynik < 0 z powrotem dodaj dzielnik
- Dzielenie znakowe
  - Dziel wartości bezwzględne
  - Dopasuj znaki ilorazu i reszty

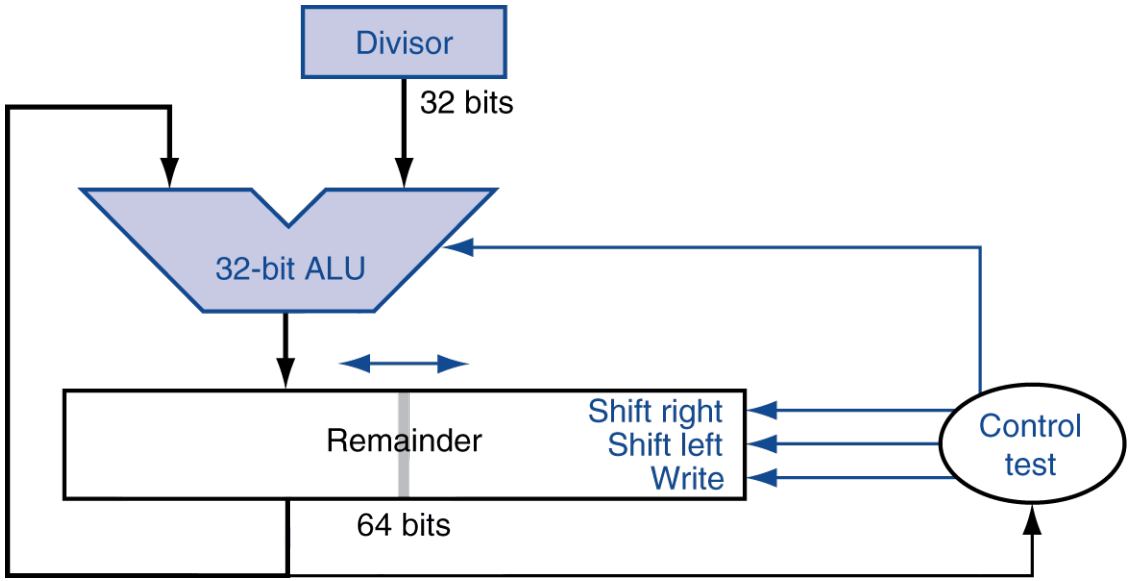
n-bitowe operandy dają *n*-bitowe iloraz i resztę

Układ dzielący





Lepszy układ dzielący



- Jeden cykl na odejmowanie dzielnika
- Przypomina układ mnożący!
  - Na obie operacje można użyć tego samego sprzętu

Szybsze dzielenie

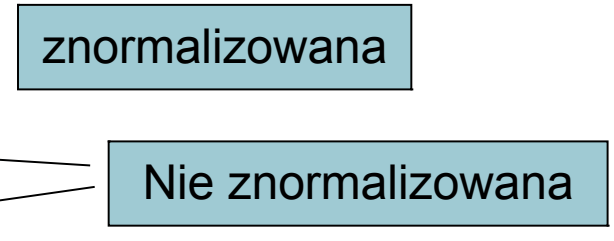
- Nie da się użyć sprzętu równoległego jak w mnożeniu
  - Odejmowanie jest warunkowe, zależy od znaku reszty
- Szybkie układy (np. dzielniki SRT) generują wiele bitów ilorazu w jednym kroku
  - Jednak pracują w wielu krokach

Dzielenie w ARM'ie

- W klasycznym ARM'ie nie ma!
- W ARMv7R i ARMv7M są instrukcje
  - `SDIV / UDIV`
  - Czyli dzielenie znakowe/bezznakowe

Zmienny przecinek

- Reprezentacja dla liczb niecałkowitych
  - W tym bardzo małych i bardzo dużych
- Podobna do notacji naukowej
  - -2.34×10^{56}
  - $+0.002 \times 10^{-4}$
  - $+987.02 \times 10^{9}$
- Binarnie
  - $\pm 1.xxxxxxx_2 \times 2^{yyyy}$
- Typy `float` i `double` w C



Standard

- Zdefiniowany przez IEEE Std 754-1985
- Opracowany w reakcji na rozbieżność reprezentacji
  - Problemy z przenaszalnością programów naukowych
- Obecnie niemal powszechnie przyjęty
- Dwie reprezentacje
  - Pojedyncza dokładność (32-bitowa)
  - Podwójna dokładność (64-bitowa)

Format IEEE Floating-Point

pojedynczy: 8 bitów pojedynczy: 23 bity
podwójny: 11 bitów podwójny: 52 bity

S	Wykładnik	Mantysa (Fraction)

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- S: bit znaku (0 ⇒ nieujemny, 1 ⇒ ujemny)
- Normalizuj mantysę (significand): 1.0 ≤ |mantysa| < 2.0
  - Zawsze ma jedynkę przed przecinkiem, tak więc nie ma potrzeby jej jawnego reprezentowania (ukryty bit)
  - „Significand” to jest „Fraction” (ułamek) z odtworzonym “1.”
- Wykładnik (exponent): reprezentacja z przesunięciem: faktyczny wykładnik + przesunięcie (bias)
  - Wykładnik jest więc bezznakowy
  - Format pojedynczy: Bias = 127; Podwójny: Bias = 1203

Zakres pojedynczej precyzji

- Wykładniki 00000000 i 11111111 są zarezerwowane
- Najmniejsza wartość dodatnia
  - Wykładnik: 00000001
    ⇒ faktyczny wykładnik = 1 – 127 = –126
  - Mantysa: 000…00 ⇒ = 1.0
  - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Największa wartość
  - Wykładnik: 11111110
    ⇒ faktyczny wykładnik = 254 – 127 = +127
  - Mantysa: 111…11 ⇒ ≈ 2.0
  - $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

Zakres podwójnej precyzji

- Wykładniki 0000…00 i 1111…11 są zarezerwowane
- Najmniejsza wartość dodatnia
  - Wykładnik: 00000000001
    ⇒ faktyczny wykładnik = 1 – 1023 = –1022
  - Mantysa: 000…00 ⇒ = 1.0
  - $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
- Największa wartość
  - Wykładnik: 11111111110
    ⇒ faktyczny wykładnik = 2046 – 1023 = +1023
  - Mantysa: 111…11 ⇒ ≈ 2.0
  - $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

Dokładność

- Dokładność względna
  - Wszystkie bity mantysy są znaczące
  - Pojedyncza: około 2^{-23}
    - Równoważne $23 \times \log_{10}2 \approx 23 \times 0.3 \approx 6$ cyfrom dziesiętnym precyzji
  - Podwójna: około 2^{-52}
    - Równoważne $52 \times \log_{10}2 \approx 52 \times 0.3 \approx 16$ cyfrom dziesiętnym precyzji

Przykład

- Reprezentacja –0.75
  - $-0.75 = (-1)^1 \times 1.1_2 \times 2^{-1}$
  - S = 1
  - Mantysa = $1000\ldots00_2$
  - Wykładnik = –1 + przesunięcie
    - Pojedyncza: $-1 + 127 = 126 = 01111110_2$
    - Podwójna: $-1 + 1023 = 1022 = 01111111110_2$
- Pojedyncza: 10111111101000…00
- Podwójna: 10111111111101000…00

Przykład

- Jaka liczba jest reprezentowana przez liczbę zmiennoprzecinkową pojedynczej precyzji

  1100000010100 0…00

  - S = 1
  - Mantysa = $01000\ldots00_2$
  - Wykładnik = $10000001_2 = 129$

- $x = (-1)^1 \times (1 + 01_2) \times 2^{(129 - 127)}$

  $= (-1) \times 1.25 \times 2^2$

  $= -5.0$

Dodawanie

- Przykład: 4 cyfry dziesiętne
  - $9.999 \times 10^{1} + 1.610 \times 10^{-1}$
- 1. Wyrównaj przecinki
  - Przesuń liczbę z mniejszym wykładnikiem
  - $9.999 \times 10^{1} + 0.016 \times 10^{1}$
- 2. Dodaj mantysy
  - $9.999 \times 10^{1} + 0.016 \times 10^{1} = 10.015 \times 10^{1}$
- 3. Normalizuj wynik i sprawdź czy przepełnienie/niedobór
  - 1.0015×10^{2}
- 4. Zaokrąglij i normalizuj jeśli trzeba
  - 1.002×10^{2}

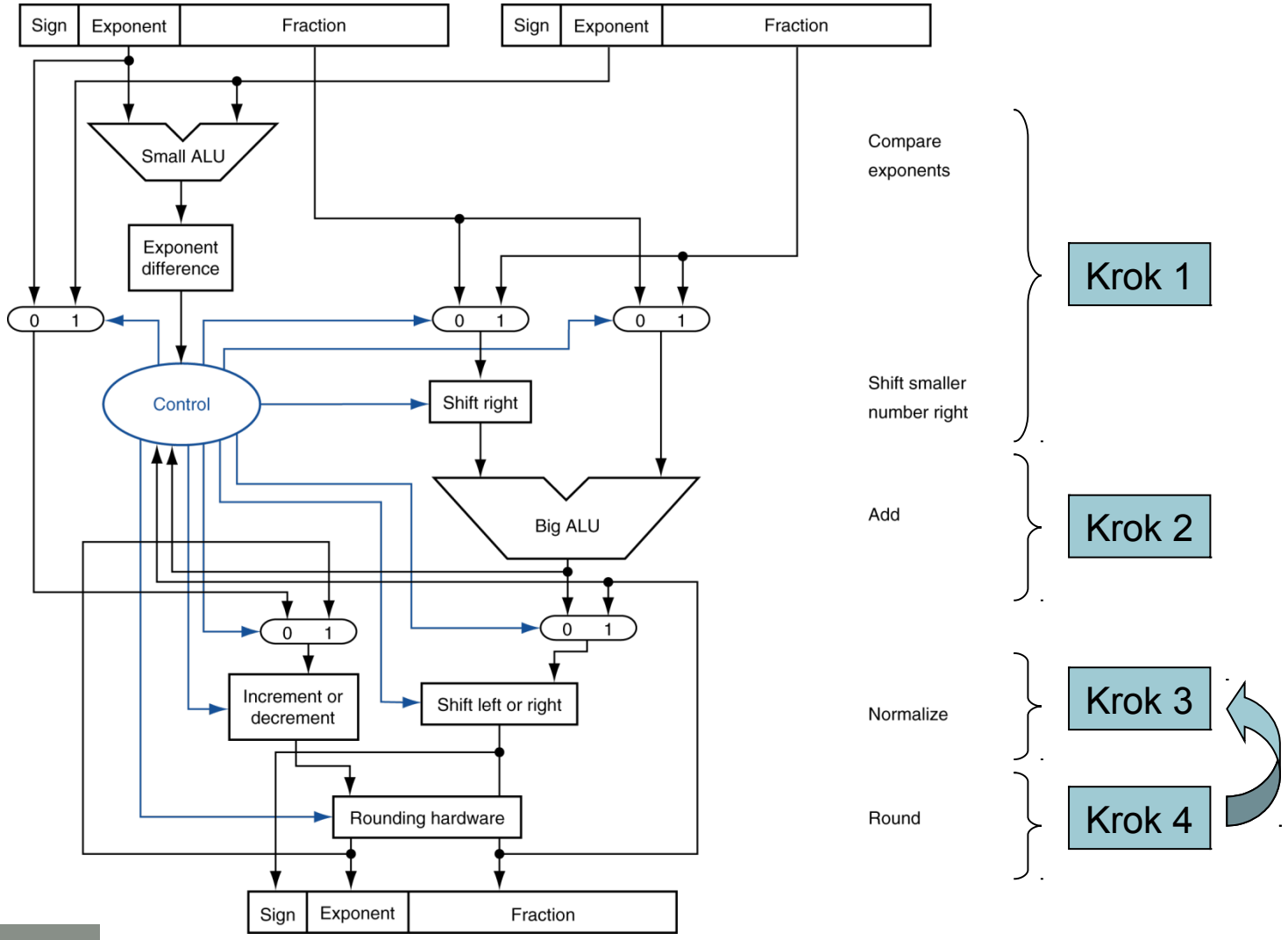
Dodawanie

- Przykład: 4 cyfry binarne
  - $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ (0.5 + –0.4375)
- 1. Wyrównaj przecinki
  - Przesuń liczbę z mniejszym wykładnikiem
  - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$
- 2. Dodaj mantysy
  - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$
- 3. Normalizuj wynik i sprawdź czy przepełnienie/niedobór
  - $1.000_2 \times 2^{-4}$
- 4. Zaokrąglij i normalizuj jeśli trzeba
  - $1.000_2 \times 2^{-4}$ (b.z.) = 0.0625

Sprzęt sumatora

- Znacznie bardziej złożony od sumatora stałoprzecinkowego
- Wykonywanie w jednym cyklu mogłoby trwać zbyt długo
  - Trwa znacznie dłużej niż operacje stałoprzecinkowe
  - Wolniejszy zegar mógłby spowolnić wszystkie instrukcje
- Sumator FP działa zwykle wiele cykli
  - Może zawierać potok

Sprzęt sumatora



Sprzęt układu mnożącego

- Zmiennoprzecinkowy układ mnożący ma złożoność zbliżoną do złożoności sumatora zmiennoprzecinkowego
  - Ale używa układu mnożącego do mantys zamiast sumatora
- Zmiennoprzecinkowy sprzęt zwykle
  - Dodaje, odejmuje, mnoży, dzieli, oblicza odwrotność, pierwiastek kwadratowy
  - Konwersja FP ↔ integer
- Operacje zwykle trwają kilka cykli
  - Mogą być wykonywane potokowo

Instrukcje FP ARM'a

- Sprzęt FP to koprocesor
  - Dodatkowy procesor rozszerzający ISA
- Oddzielne rejestry FP
  - 32 pojedynczej precyzji: s0, s1, … s31
  - Sparowane dla podwójnej precyzji: d0=s0/s1, …
    - Vector FP (VFP) koprocesor ma 32 × 64-bit FP rejestrów
- Instrukcje FP działają tylko na rejestrach FP
  - Programy na ogół nie wykonują operacji stałoprzecinkowych na danych FP i vice versa
  - Więcej rejestrów a minimalny wpływ na wielkość kodu
- FP instrukcje load i store (mniej trybów adresowania, 3 rodzaje immediate offset)
  - FLDS, FLDD, FSTS, FSTD
    - np. `FLDS s8, [sp,#32]`

Instrukcje FP ARM'a

- Arytmetyka pojedynczej precyzji
  - FADDS, FSUBS, FMULS, FDIVS
    - np. `FADDS s0, s1, s6`
- Arytmetyka podwójnej precyzji
  - FADDD, FSUBD, FMULD, FDIVD
    - np. `FMULD d4, d4, d6`
- Wersje FP instrukcji porównania
  - FCMPS, FCMPD
  - Przed skokiem bity warunków FP trzeba kopiować do bitów warunków stałoprzecinkowych
    - FMSTAT
  - Zmiana ról niektórych instrukcji skoku:
    - `BLT => BMI, BLE => BLS`

Przykład FP: °F na °C

- Kod C:

```
float f2c (float fahr) {
  return ((5.0/9.0)*(fahr - 32.0));
}
```

  - `fahr` w s12, wynik w s0, stałe są „w zasięgu” r12

- Skompilowany kod ARM’a:

```
f2c: FLDS  s16, [r12, const5]
     FLDS  s18, [r12, const9]
     FDIVS s16, s16, s18
     FLDS  s18, [r12, const32]
     FSUBS s18, s12, s18
     FMULS s0,  s16, s18
     MOV   pc,  lr
```

Przykład FP: Mnożenie macierzy

- X = X + Y × Z
  - Macierze 32 × 32, elementy podwójnej precyzji 64-bitowe
- Kod C:

```
void mm (double x[][],
         double y[][], double z[][]) {
  int i, j, k;
  for (i = 0; i! = 32; i = i + 1)
    for (j = 0; j! = 32; j = j + 1)
      for (k = 0; k! = 32; k = k + 1)
        x[i][j] = x[i][j]
                  + y[i][k] * z[k][j];
}
```

  - Adresy x, y, z w r0, r1, r2, a i, j, k w r3, r4, r5

Przykład FP: Mnożenie macierzy

- Kod ARM'a:

```
mm:

    SUB  sp, sp, #12          ; stos: miejsce na 3 rejestry
    STR  r4, [sp, #8]         ; zachowaj r4 na stosie
    STR  r5, [sp, #4]         ; zachowaj r5 na stosie
    STR  r6, [sp, #0]         ; zachowaj r6 na stosie
    MOV  r3, #0               ; i = 0; indeks 1-szej pętli
L1: MOV  r4, #0               ; j = 0; restart 2-giej pętli
L2: MOV  r5, #0               ; k = 0; restart 3-ciej pętli
    ADD  r6, r4, r3, LSL #5   ; r6 = i * size(row) + j
    ADD  r6, r0, r6, LSL #3   ; r6 = adres bajtowy x[i][j]
    FLDD d4, [r6, #0]         ; d4 = 8 bajtów x[i][j]
L3: ADD  r6, r4, r5, LSL #5   ; r6 = k * size(row) + j
    ADD  r6, r2, r6, LSL #3   ; r6 = adres bajtowy z[k][j]
    FLDD d8, [r6, #0]         ; d8 = 8 bajtów z[k][j]
```

Przykład FP: Mnożenie macierzy

```
…
    ADD   r6, r5, r3, LSL #5 ; r6 = i*size(row) + k
    ADD   r6, r1, r6, LSL #3 ; r6 = adres bajtowy y[i][k]
    FLDD  d9, [r6, #0]       ; d9 = 8 bajtów y[i][k]
    FMULD d8, d9, d8         ; d8 = y[i][k] * z[k][j]
    FADDD d4, d4, d8         ; d4=x[i][j]+ y[i][k]*z[k][j]
    ADD   r5, r5, #1         ; k = k + 1

    CMP   r5, #32
    BLT   L3                 ; jeśli (k != 32) skocz do L3
    FSTD  d4, [r6, #0]       ; x[i][j] = d4
    ADD   r4, r4, #1         ; j = j + 1
    CMP   r4, #32
    BLT   L2                 ; jeśli (j != 32) skocz do L2
    ADD   r3, r3, #1         ; i = i + 1
    CMP   r3, #32
    BLT   L1                 ; jeśli (i != 32) skocz do L1
```

Dokładna arytmetyka

- IEEE Std 754 specyfikuje dodatkową kontrolę zaokrąglania
  - Dodatkowe bity precyzji (guard, round, sticky)
  - Wybór trybów zaokrąglania
  - Pozwala programiście na dostrojenie numerycznego zachowania obliczeń
- Nie wszystkie jednostki FP implementują wszystkie opcje
  - Większość języków programowania i bibliotek FP używa wersji podstawowej
- Kompromis pomiędzy złożonością sprzętu, wydajnością i wymaganiami rynkowymi

Równoległość podsłów

- W zastosowaniach graficznych i audio przydaje się wykonywanie jednoczesnych operacji na krótkich wektorach
  - Przykład: 128-bitowy sumator:
    - Szesnaście dodawań 8-bitowych
    - Osiem dodawań 16-bitowych
    - Cztery dodawania 32-bitowe
- Równoległość na poziomie danych (data-level parallelism), vector parallelism, lub Single Instruction Multiple Data (SIMD)

x86 FP Architecture

- Originally based on 8087 FP coprocessor
  - 8 × 80-bit extended-precision registers
  - Used as a push-down stack
  - Registers indexed from TOS: ST(0), ST(1), …
- FP values are 32-bit or 64 in memory
  - Converted on load/store of memory operand
  - Integer operands can also be converted on load/store
- Very difficult to generate and optimize code
  - Result: poor FP performance

x86 FP Instructions

Data transfer	Arithmetic	Compare	Transcendental
FILD mem/ST(i) FISTP mem/ST(i) FLDPI FLD1 FLDZ	FIADDP mem/ST(i) FISUBRP mem/ST(i) FIMULP mem/ST(i) FIDIVRP mem/ST(i) FSQRT FABS FRNDINT	FICOMP FIUCOMP FSTSW AX/mem	FPATAN F2XMI FCOS FPTAN FPREM FPSIN FYL2X

- Optional variations
  - I: integer operand
  - P: pop operand from stack
  - R: reverse operand order
  - But not all combinations allowed

Streaming SIMD Extension 2 (SSE2)

- Adds 4 × 128-bit registers
  - Extended to 8 registers in AMD64/EM64T
- Can be used for multiple FP operands
  - 2 × 64-bit double precision
  - 4 × 32-bit double precision
  - Instructions operate on them simultaneously
    - Single-Instruction Multiple-Data

Matrix Multiply

- Unoptimized code:

```
1. void dgemm (int n, double* A, double* B, double* C)
2. {
3.  for (int i = 0; i < n; ++i)
4.    for (int j = 0; j < n; ++j)
5.    {
6.     double cij = C[i+j*n]; /* cij = C[i][j] */
7.     for(int k = 0; k < n; k++ )
8.      cij += A[i+k*n] * B[k+j*n]; /* cij += A[i][k]*B[k][j] */
9.     C[i+j*n] = cij; /* C[i][j] = cij */
10.   }
11. }
```

Matrix Multiply

- x86 assembly code:

```
1. vmovsd (%r10),%xmm0  # Load 1 element of C into %xmm0
2. mov %rsi,%rcx        # register %rcx = %rsi
3. xor %eax,%eax        # register %eax = 0
4. vmovsd (%rcx),%xmm1  # Load 1 element of B into %xmm1
5. add %r9,%rcx         # register %rcx = %rcx + %r9
6. vmulsd (%r8,%rax,8),%xmm1,%xmm1 # Multiply %xmm1,
   element of A
7. add $0x1,%rax        # register %rax = %rax + 1
8. cmp %eax,%edi        # compare %eax to %edi
9. vaddsd %xmm1,%xmm0,%xmm0 # Add %xmm1, %xmm0
10. jg 30 <dgemm+0x30>  # jump if %eax > %edi
11. add $0x1,%r11d      # register %r11 = %r11 + 1
12. vmovsd %xmm0,(%r10) # Store %xmm0 into C element
```

Matrix Multiply

- Optimized C code:

```
1. #include <x86intrin.h>
2. void dgemm (int n, double* A, double* B, double* C)
3. {
4.  for ( int i = 0; i < n; i+=4 )
5.   for ( int j = 0; j < n; j++ ) {
6.    __m256d c0 = _mm256_load_pd(C+i+j*n); /* c0 = C[i]
   [j] */
7.    for( int k = 0; k < n; k++ )
8.     c0 = _mm256_add_pd(c0, /* c0 += A[i][k]*B[k][j] */
9.               _mm256_mul_pd(_mm256_load_pd(A+i+k*n),
10.              _mm256_broadcast_sd(B+k+j*n)));
11.   _mm256_store_pd(C+i+j*n, c0); /* C[i][j] = c0 */
12.  }
13. }
```

Matrix Multiply

- Optimized x86 assembly code:

```
1. vmovapd (%r11),%ymm0       # Load 4 elements of C into %ymm0
2. mov %rbx,%rcx              # register %rcx = %rbx
3. xor %eax,%eax              # register %eax = 0
4. vbroadcastsd (%rax,%r8,1),%ymm1 # Make 4 copies of B element
5. add $0x8,%rax              # register %rax = %rax + 8
6. vmulpd (%rcx),%ymm1,%ymm1 # Parallel mul %ymm1,4 A elements
7. add %r9,%rcx               # register %rcx = %rcx + %r9
8. cmp %r10,%rax              # compare %r10 to %rax
9. vaddpd %ymm1,%ymm0,%ymm0   # Parallel add %ymm1, %ymm0
10. jne 50 <dgemm+0x50>       # jump if not %r10 != %rax
11. add $0x1,%esi             # register % esi = % esi + 1
12. vmovapd %ymm0,(%r11)      # Store %ymm0 into 4 C elements
```

Przesunięcie w prawo

- Przesunięcie w lewo o *i* miejsc mnoży liczbę całkowitą przez 2^i
- Przesunięcie w prawo dzieli przez 2^i?
  - Tylko dla liczb bezznakowych
- Dla liczb znakowych
  - Arytmetyczne przesunięcie w prawo: powtarza bit znaku
  - np. –5 / 4
    - 11111011_2 >> 2 = 11111110_2 = –2
    - Zaokrągla w kierunku –∞
  - por. 11111011_2 >>> 2 = 00111110_2 = +62

Łączność

- Programy równoległe mogą przeplatać operacje w nieoczekiwanej kolejności
  - Założenie łączności może być fałszywe

		(x+y)+z	x+(y+z)
x	-1.50E+38		-1.50E+38
y	1.50E+38	0.00E+00	
z	1.0	1.0	1.50E+38
		1.00E+00	0.00E+00

- Konieczność sprawdzania programów równoległych w różnych sytuacjach

Kto dba p dokładność FP?

- Istotne dla programów naukowych
  - Ale czy dla zwykłego konsumenta?
    - “Mój bilans bankowy ma błąd 0.0002¢!” ☹
- The Intel Pentium FDIV bug
  - Rynek oczekuje dokładności
  - Patrz Colwell, *The Pentium Chronicles*

Uwagi końcowe

- Bity nie mają określonego znaczenia
  - Ich interpretacja zależy od zastosowanej instrukcji
- Reprezentacja liczb w komputerze
  - Skończony zakres i dokładność
  - Należy się z tym liczyć przy programowaniu

Uwagi końcowe

- Arytmetyka ISA komputerów
  - Znakowe i bezznakowe liczby całkowite
  - Zmiennoprzecinkowa aproksymacja liczb rzeczywistych
- Ograniczony zakres i dokładność
  - Może powstać przepełnienie i niedobór
- ARM ISA
  - Instrukcje najczęściej używane (core): ok. 40
    - 100% w SPECINT, 97% w SPECFP
  - Pozostałe instrukcje są rzadziej używane